אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

Model based deep learning project:

Levenberg-Marquardt algorithm unfolded

Rachel ziderman
211723887

Ofek luria
207961574

24/07/2025

A. Motivation and background

Brillouin microscopy is an advanced optical technique that enables non-contact mechanical characterization of materials, based on analysis of the Brillouin spectrum generated by light scattering within a sample. Accurate analysis of this spectrum is essential for extracting key mechanical parameters. The spectrum is modulated using Lorentzian function with a single peak hight, line width and central frequency for homogenous matter.

Accurate estimation of the spectrum parameters is essential for the Brillouin microscopy field, since the parameters can be directly linked to the mechanical properties of the sample.

Currently, various iterative approaches are used to solve the least squares problem in order to fit a curve to the Lorentzian model, including Levenberg-Marquardt algorithm which is used as default in *scipy.optimize.curve_fit*

In this project we wish to unfold the LM algorithm and try to improve its MSE criterion.

B. Problem formulation

In this project we consider a simpler case – a gaussian spectrum model, contaminated by white gaussian noise. The Gaussian spectrum profile analysis comes as a first step in the research field thanks to its smoother nature, while still acts in many ways like the Lorentzian profile.

For those reasons, we chose our data model to be gaussian:

$$y(\omega) = \frac{A}{\sqrt{2\pi \cdot \sigma^2}} \cdot exp\left(\frac{-(\omega - \mu_0)^2}{2 \cdot \sigma^2}\right) + \mathrm{v}(\omega),\ \mathrm{v} \sim N\left(0, {\sigma_N}^2\right)$$

With the discussed parameters being:

$$\theta^o = \{A, \sigma, \mu_0\}$$

We wish to solve the following non-linear least squares problem:

$$\theta^{o*} = \arg\min_{\theta^o} \ \|y - f\ (\omega, \theta^o)\|^2$$

The non-linearity comes from the function $f(x, \theta^o)$ being gaussian function:

$$f(\omega, \theta^o) = \frac{A}{\sqrt{2\pi \cdot \sigma^2}} \cdot exp\left(\frac{-(\omega - \mu_0)^2}{2 \cdot \sigma^2}\right)$$

The iterative solver chosen for this task is the Levenberg-Marquardt algorithm (LM) with the following update rule:

$$\widehat{\theta^o}_{k+1} = \widehat{\theta^o}_k - (J^T J + \lambda_k I)^{-1} \cdot\ J^T \cdot \left(y - f\ (x, \widehat{\theta^o}_k)\right)$$

$$J = \frac{df\left(f(x, \theta^o)\right)}{d\theta^o}\Big|_{\theta_k^o}$$

$$\lambda_k = g(\lambda_{k-1}, \alpha),\ \ where\ g(x, \alpha) = \begin{Bmatrix} x \cdot \alpha, & if(condition) \\ \dfrac{x}{\alpha}, & if(not\ condition) \end{Bmatrix}$$

And the condition is if the a-posterior error is smaller than the a-priori error.

The error defined as:

$$e_k = y(\omega) - f(\omega, \widehat{\theta^o}_k)$$

The lambda update condition is then defined to be:

$$condition = |e_{k+1}|^2 - |e_k|^2 > 0$$

This update rule is based on the principle that if an iteration brings the parameters closer to the target, the next update becomes more conservative. Conversely, if the parameters move further away from the target, the subsequent update becomes more aggressive.

For our vanilla algorithm implementation, we used $\alpha = 10$.

C. Methods

In the task of improving the LM algorithm we choose the unfolding strategy of the first kind. In this method we fix the number of iterations. We consider α parameter being a learned parameter that can be optimized using data.

The implementation took the following form:

- Date:
  training set and test set of generated date. Each data sample is a gaussian profile with additive white gaussian noise, over x grid from -4 to 4, with 100 samples.
  Generated data with randomized θ^o with the following distributions:

$$\mathrm{A} \sim U[1,10]$$

$$\mu \sim U[-1,\ 1]$$

$$\sigma \sim U[\ 0.1,\ 1]$$

$$\sigma_N = 0.5$$

- Model:
  We define the learned LM model to be inherited from nn.Module.
  The empty model has a pre-defined number of iterations T and (T-1) unique $\alpha_t\ \ for\ t = 1, 2\ \ldots T-1$. At these setting we can consider each iteration as a layer of neuron network that can be backpropagated through in order to optimize α

  The forward path the model implements is almost the vanilla LM algorithm, with slight change in the way λ updates after each iteration. These changes are necessary to keep the algorithm end-to-end differentiable. (further discussion later)

  The model outputs a set of estimated parameters, that are compared to the true parameters through MSE loss function. The loss function is differentiated with respect to each α_t

- End-to-end differentiability:
  We are now interested at optimizing α , therefore we need end to end differentiability with respect to α

  The update rule of λ_t was changed, in order to keep the relation with λ_{t-1} differentiable:

$$\lambda_t = \alpha_{t-1}^{\tanh\left(10\cdot(e_p - e_a)\right)} \cdot \lambda_{t-1}$$

  We are now interested in computing the following term:

$$\frac{\partial \theta_{k+1}^{o}}{\partial \alpha_n} = \frac{\partial \theta_{k+1}^{o}}{\partial \theta_k^{o}} \cdot \frac{\partial \theta_k^{o}}{\partial \theta_{k-1}^{o}} \cdot \ldots \frac{\partial \theta_{n+1}^{o}}{\partial \lambda_n} \cdot \frac{\partial \lambda_n}{\partial \alpha_n} \quad for\ n = 1,2 \ldots, k$$

$\frac{\partial \lambda_n}{\partial \alpha_n}$: this term exists and finite because the relation of λ_t and λ_{t-1}

$$\frac{\partial \theta_{n+1}^{o}}{\partial \lambda_n} = \frac{\partial}{\partial \lambda_n} [\theta_n^{o} - (J^T J + \lambda_n I)^{-1} \cdot J^T \cdot \left(y - f\left(x, \theta_n^{o}\right)\right)] =$$

$$= (J^T J + \lambda_n I)^{-2} \cdot J^T \cdot (y - f(x, \theta_n^{o}))$$

$\frac{\partial \theta_k^{o}}{\partial \theta_{k-1}^{o}}$: this term exists and finite because it involves derivatives of smooth function: J is the Jacobian of gaussian function with respect to its 3 parameters, and f is just a gaussian function.

And thus, with the smooth update rule for λ the algorithm is end-to-end differentiable.

Note: At the beginning of the project, we chose to build the learned algorithm using a unique λ for each iteration, where λ was the sole trainable parameter optimized during training.
When we implemented this model and compared it to the vanilla version, which included a dynamic λ update based on a factor and a condition on the posterior error, we observed that the vanilla model outperformed the learned model with a fixed and optimized λ.

This led us to understand that updating λ based on the posterior error condition is a stronger mechanism than learning fix λ for each iteration. Therefore, we revised our approach to optimize the model using a λ that changes dynamically through a factor. Our new approach included optimizing the update factor α, as described above.

D. Results

Convergence example:

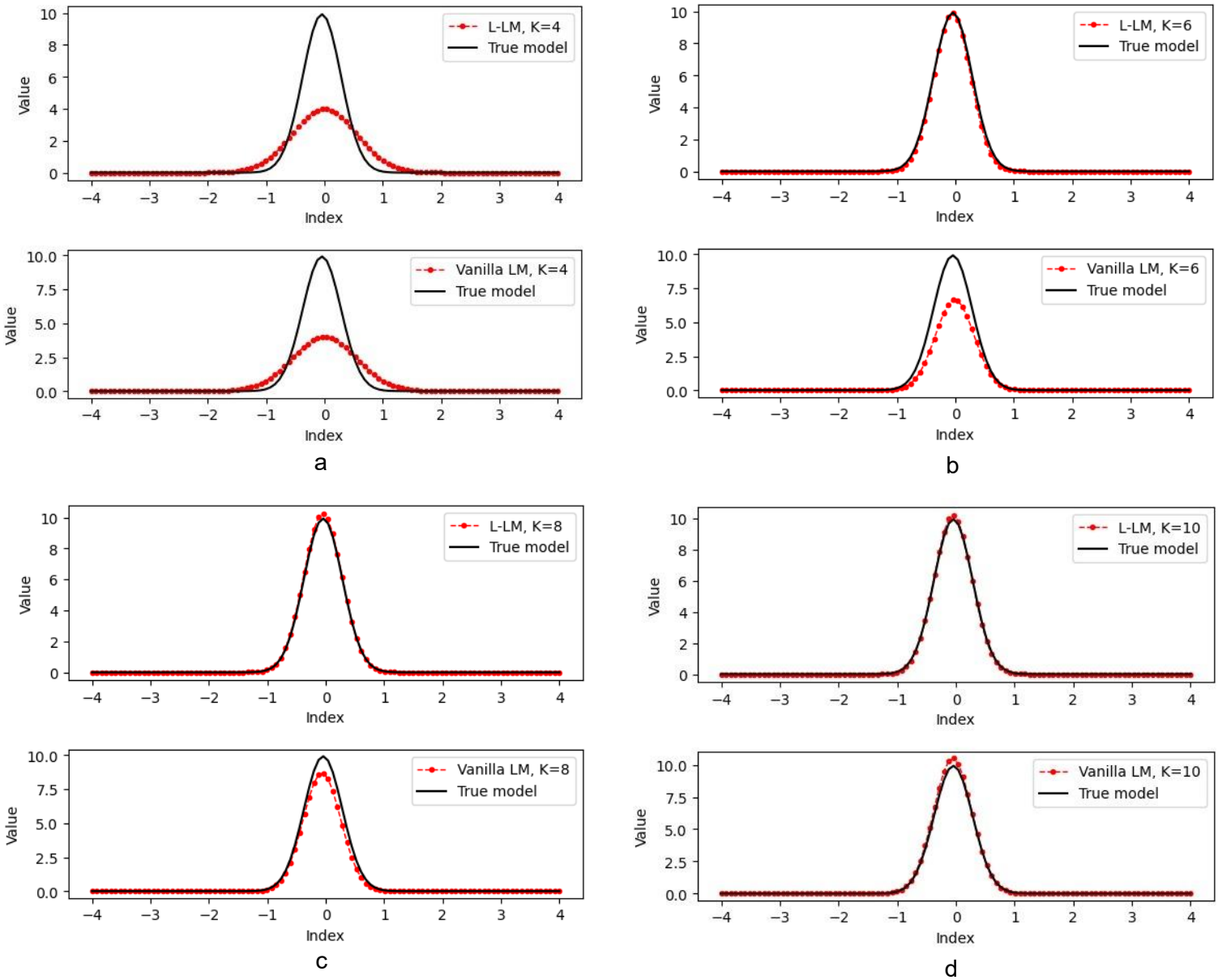

Figure 1: Convergence example of vanilla LM Vs. learned LM

This figure presents the convergence of the two algorithms for a single data sample, as a function of the number of iterations. It can be seen that the learned algorithm outperformed the vanilla LM. Although the vanilla LM also converges in small number of iterations.

MSE Comparison: standard vs. Learned Model

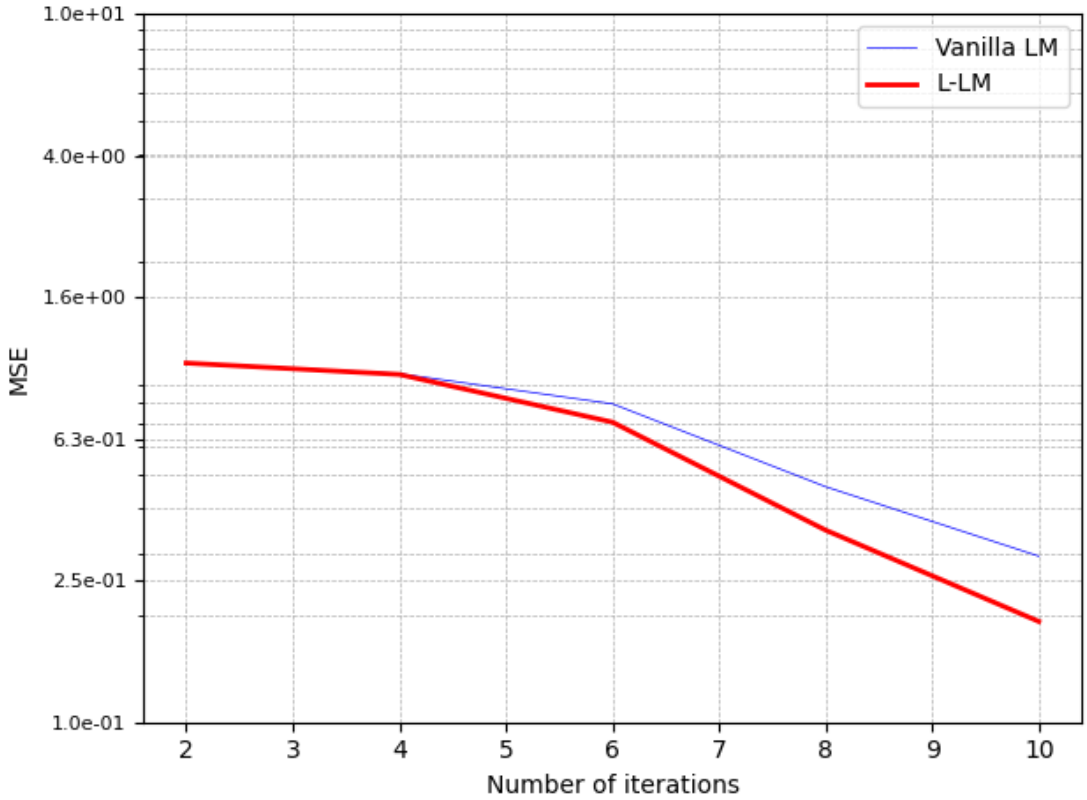

Figure 2: MSE of the two models

This figure shows that the learned LM algorithm demonstrates improved performance in terms of MSE, compared to the vanilla LM.

This result shows that the goal we were aiming too, which is improvement in the algorithm performance in terms of MSE was achieved by using the unfolding method.

Though, the improvement isn't as sufficient as we would expect it to be and both algorithms exhibit a similar decreasing trend.

Noise level:

Next, we will like to examine the models preformances for different noise level.

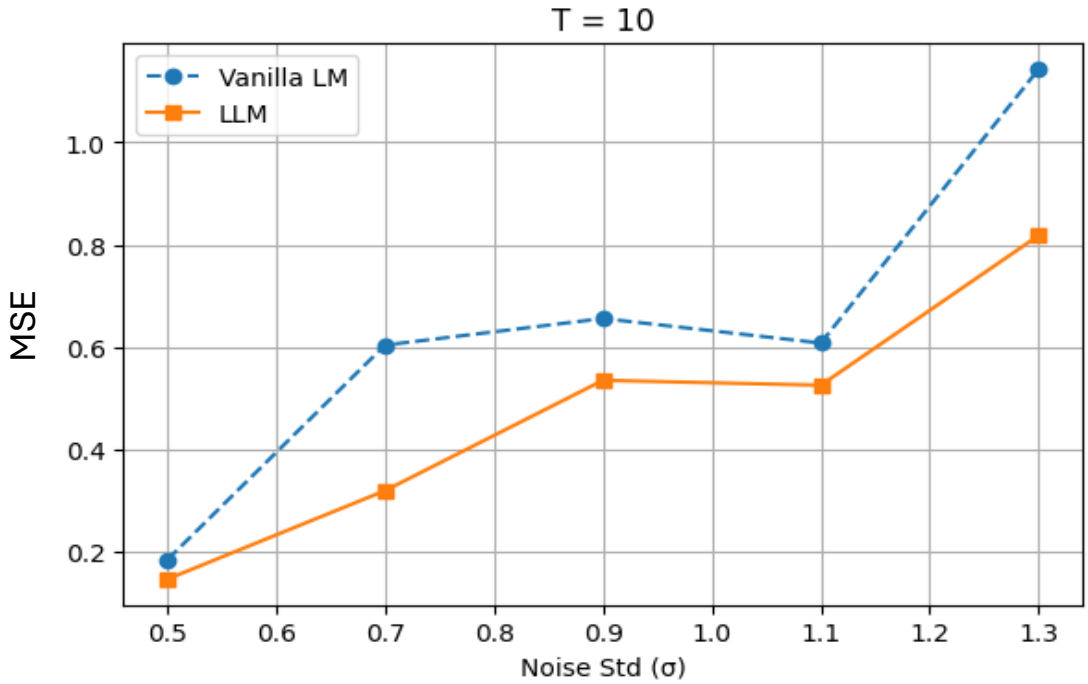

Figure 3: MSE performance of the models for different noise level

This figure shows that the learned model has no significant advantage in dealing with different noise levels.

E. Discussion

This project aim was to improve the performance of LM algorithm by using an unfolding method and optimize the hyper parameter α that controls the update of the regularization factor λ per iteration.

The main challenge was optimizing a parameter that already actively change as function of the a-posteriori error after each step.
this led to 2 main problems: 1. The vanilla with the active change outperforms a learned unique λ_t . 2. The update rule has hard "if" statement that ruins differentiability.

We solved those problems by using a soft update rule with differentiable function (tanh(x)) and optimizing the parameter α.

We than manage to get a slightly better performing algorithm in term of MSE, but not much improvement was observed.